Coordinated Wildfire Inspection with a Battery-Constrained Drone Fleet: DQN, REINFORCE and A2C Compared

Lithma Perera
Student ID 24101822
Curtin University

Sulakna Weerasinghe
Student ID 24105235
Curtin University

Sayuru Bopitiya
Student ID 24071158
Curtin University

***Abstract*—Emergency responders need fire fronts and high-priority areas to be inspected again and again, but a small fleet of battery-limited drones must also return to share a single charger. We model this as a fully observable, finite-horizon Markov decision process in which one central controller chooses a joint move for three drones on a 40×40 grid with stochastic fire spread and changing wind. Three reinforcement learning (RL) agents were implemented and compared: a deep Q-network (DQN), REINFORCE with a learned baseline, and synchronous advantage actor–critic (A2C). Each was trained with three random seeds and evaluated on 20 unseen worlds against a one-step Greedy controller, a rolling-horizon planner, a lawnmower sweep and random policies. After a feasibility mask was added, every learned policy kept all drones alive, but inspection quality stayed at the level of random feasible actions: weighted freshness of 0.084 (DQN), 0.069 (REINFORCE) and 0.084 (A2C) against 0.084 for masked random and 0.221 for Greedy. A learning-rate sweep and a sensitivity study did not change this picture. Measurements show that the immediate reward separates good and bad joint actions by far less than the noise in mission returns, which explains why none of the agents learned coordinated patrols. We propose factorised credit assignment, potential-based shaping and learning from demonstrations as improvements, and partial observability, decentralised communication and calibrated fire physics as extensions.**

***Index Terms*—reinforcement learning, wildfire monitoring, multi-drone coordination, Markov decision process, DQN, policy gradient, actor–critic, action masking**

PART I: DESCRIPTION AND CRITICAL REVIEW

I. INTRODUCTION

(Written by Lithma Perera)

Extreme wildfires cause loss of life, property and ecosystems on every vegetated continent, and a warming climate is making the conditions for them more common [1]. During a response, incident teams need up-to-date, close-range views of the active fire front and of assets at risk such as roads, settlements and power lines. Unmanned aerial vehicles (UAVs) are increasingly used for this task because they are cheap, fast to deploy and safe to fly over hazardous terrain [2]. A single drone, however, has limited endurance. A small fleet can cover more ground, but only if its members coordinate: they must spread out over the areas that most need attention, return before their batteries run out, and take turns at a shared charging point.

This project studies that coordination problem in simulation. Three battery-limited drones are commanded by one central controller. Fires spread stochastically under changing wind, and every location has an inspection age that grows until a drone flies over it. The goal is to keep high-priority locations, especially burning cells and the fire front, recently inspected for the whole mission while never losing a drone.

The problem is hard for three reasons. First, decisions are joint: five commands per drone give $5^3 = 125$ joint actions at every step, and the value of one drone's move depends on where the other two are. Second, consequences are delayed: flying too far from base costs nothing immediately but strands the drone many steps later. Third, one charger creates contention, so the fleet must stagger its returns. This is a sequential decision problem under uncertainty with a numerical objective, the setting reinforcement learning (RL) is designed for [3]. Prior work has applied deep RL to wildfire surveillance with autonomous aircraft [4] and to fire-front monitoring with multiple UAVs [5]. Unlike those studies, we give every controller the full state, so that RL can be compared fairly with conventional planning that uses the same information.

We contribute a Gymnasium environment for this task, three agents trained under one fixed protocol, a comparison on unseen worlds against conventional controllers with tuning and sensitivity studies, and a measured explanation of why the agents did not learn. The code, executed notebook and archived results accompany this report [6].

II. MARKOV DECISION PROCESS AND ENVIRONMENT

(Written by Lithma Perera)

We formulate the task as a finite-horizon Markov decision process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, H)$ with horizon $H = 300$ steps and discount $\gamma = 0.99$ [3]. The environment follows the Gymnasium `Env` interface. All values below are those of the 40×40 main study; the earlier 20×20 development study used a smaller, shorter version (horizon 160, battery 80).

A. State and observation

The state is fully observed. The observation is a dictionary of three arrays, all scaled to $[0, 1]$:

- five 40×40 maps: burning cells, remaining fuel, terrain flammability, static priority and inspection age;
- a 3×4 array with each drone's row, column, battery and a lost flag;
- 12 global values: wind direction (one-hot over calm and four directions), charger owner (one-hot over none and three drones), base position and the fraction of the mission left.

Spatial maps preserve the neighbourhood structure that convolutional networks exploit in pixel-based RL [7]. Every quantity that affects the next-state distribution, including inspection ages and remaining time, is included, so the Markov property holds without an observation history. Real fire is only partly visible; Section X returns to this. Before the network, the drone and base coordinates are also drawn as four binary maps, giving nine spatial channels and 24 vector inputs.

B. Action space

The action $a_t \in \{0, \dots, 124\}$ encodes one of five commands (stay, north, south, west, east) for each drone. A single joint action lets a central controller coordinate the fleet directly and lets us use standard discrete-action algorithms without changing them. Its cost is a size that grows as 5^N with fleet size N; Section IX discusses factorised alternatives. Inspection is automatic: every cell within a Chebyshev radius of two cells of an active drone is inspected after it moves.

C. Transition model

Each step applies the following updates in order, with all randomness drawn from the world's seeded generator.

1) *Movement.* Moves off the grid are ignored. If two drones would enter the same non-base cell or swap cells, both moves are rejected and counted as collision attempts.
2) *Energy and charging.* A move costs 1 unit and hovering 0.25 from a capacity of $B = 160$. One drone staying at the base charges by 8 units per step; when several wait, the one with the lowest battery is served first. A drone whose battery reaches zero away from base is lost permanently.
3) *Inspection.* All ages increase by one; cells under a sensor footprint are reset to zero.
4) *Fire and wind.* A burning cell burns one unit of fuel per step and goes out when its fuel is exhausted. An unburnt cell i with fuel ignites with probability

$$P_i = 1 - \prod_{j \in \mathcal{N}_i \cap \mathcal{F}_t} \big(1 - \min(1,\, q\, \omega_{ji}\, \tau_i)\big), \qquad (1)$$

where $\mathcal{N}_i$ are the four neighbours of i, $\mathcal{F}_t$ the burning cells, $q = 0.04$ the spread probability, $\tau_i \in [0.4, 1]$ the terrain flammability, and $\omega_{ji} = 2$ when the spread from j to i is downwind and 1 otherwise. The wind changes to a random direction with probability 0.06 per step.

An episode ends at the horizon or when all three drones are lost. Each world fixes the terrain, fuel, three high-priority zones and three initial fires. The physical constants are illustrative rather than calibrated.

D. Reward design

Inspection quality is defined with priority weights $w_i = p_i(1 + 2b_i + f_i)$, where $p_i \in \{1, 3\}$ marks priority zones, b_i burning cells and f_i cells on the fire front. With inspection age a_i and deadline $D = 30$, the *weighted freshness* and *overdue fraction* are

$$F_t = \frac{\sum_i w_i \max(0, 1 - a_i/D)}{\sum_i w_i}, \quad O_t = \frac{\sum_i w_i\, \mathbb{I}[a_i > D]}{\sum_i w_i}. \qquad (2)$$

The reward is

$$r_t = G_t - 0.1\, O_t - 0.02\, \frac{E_t}{B} - 0.2\, K_t - 5\, L_t - 0.1\, (H - t - 1)\, \mathbb{I}[\text{fleet lost}], \qquad (3)$$

where $G_t = \sum_{i \in C_t} w_i \min(1, a_i/D) / \sum_i w_i$ is the weighted staleness removed by this step's inspections C_t, E_t the energy used, K_t the collision attempts and L_t the newly lost drones.

Each term targets part of the goal. The inspection gain G_t is dense: it rewards a useful inspection immediately rather than only at the end of a 300-step mission, which shortens the credit-assignment path and should speed up learning. Because it is larger for stale and high-weight cells, it favours revisiting the fire front rather than circling one area. The overdue term keeps pressure on every location, the energy term discourages pointless movement, and the collision term penalises blocked commands. The loss penalty and the mission-failure term charge the remaining horizon as fully overdue, so losing the fleet early is never better than flying the whole mission. All terms are part of the task objective rather than added shaping, so they define the optimal policy instead of altering it; shaping that is not potential-based can change which policy is optimal [8]. For evaluation we report F_t and O_t averaged over the full horizon, return, energy, drones lost and decision time.

III. Reinforcement Learning Agents

(Written by Sulakna Weerasinghe)

A. Choice of agents

We chose one agent from each of the three main families of model-free RL, so that a failure could not be blamed on one family's weaknesses. DQN is value-based and off-policy: its replay buffer reuses each transition many times, which suits an expensive simulator with a discrete action set [7]. REINFORCE is a Monte Carlo policy-gradient method: it gives unbiased gradients of the expected return and learns a stochastic policy directly [9]. A2C is an actor–critic method: its critic bootstraps value estimates to reduce the variance of the policy gradient and allows updates within an episode [10]. Comparing them shows whether value estimation, full returns or bootstrapping copes best with long missions and a large joint action space.

All three share one network: two strided convolutions (16 and 32 filters) over the nine spatial channels, concatenated with the 24 vector inputs, a 128-unit hidden layer and a 125-way output, which gives Q-values for DQN and logits for the policies. REINFORCE and A2C add a scalar value head

(435,006 parameters in total at 40×40). All agents use Adam, $\gamma = 0.99$ and gradient-norm clipping at 0.5.

B. Update rules

DQN minimises the Huber loss between $Q_\theta(s,a)$ and the target $y = r + \gamma \max_{a' \in \mathcal{A}_f(s')} Q_{\bar{\theta}}(s', a')$, where $Q_{\bar{\theta}}$ is a target network copied every 100 steps and $\mathcal{A}_f$ the feasible actions. It samples minibatches of 32 from a 2,000-transition replay buffer and explores with ϵ decayed from 1 to 0.05 over 40,000 steps.

REINFORCE with a learned baseline updates once per episode with $\nabla_\theta \sum_t \gamma^t \log \pi_\theta(a_t|s_t)\,(G_t - V_\phi(s_t))$, where G_t is the discounted reward-to-go and the baseline V_ϕ is fitted by squared error.

A2C updates every 32 steps using n-step returns bootstrapped from the critic, the advantage $A_t = G_t^{(n)} - V_\phi(s_t)$, a value loss weighted 0.5 and an entropy bonus of 0.05. It is the synchronous, single-environment form of the method in [10]. Full update equations are given in the Appendix.

C. Feasibility mask

Unrestricted agents lost most of the fleet (Section IV), while the conventional controllers, which only propose safe moves, kept theirs. We therefore gave the learners the same feasibility rule. A move is feasible only if the drone keeps enough battery to fly back to base afterwards; a drone low on battery must move towards base; a drone at base below full charge must stay and charge; and joint actions that would collide are excluded. In the rare case where no joint action remains, an emergency rule moves the drones with the smallest margin home first. The mask is applied when acting and inside the DQN target, and infeasible logits are set to the lowest finite value so they receive zero probability. This follows shielding in safe RL [11] and invalid-action masking in policy-gradient methods, which keeps the policy gradient valid [12]. At reset, 73 of the 125 joint actions are feasible.

IV. Development at 20×20

(Written by Sulakna Weerasinghe)

We first developed on a 20×20 grid (horizon 160, battery 80, sensor radius 1). Without the mask, 1,000-episode runs on a GPU lost most drones, and DQN's Q-values grew to about 74,000 against a true return bound of about 80. A lower learning rate removed the blow-up, but Double DQN [13] and slower exploration did not stop the losses. Table I compares the masked and unmasked agents trained for 300 episodes with three seeds each and tested on 20 unseen worlds.

The mask was decisive for survival: all masked agents kept every drone in every mission, but inspection did not improve. Masked DQN (0.146) equalled masked random (0.145), and Greedy was twice as good. Networks trained to imitate Greedy matched 86–96% of its training decisions but only 9–18% on new worlds, so generalising the inspection ranking, not survival, was the bottleneck.

TABLE I
20×20 development study: test means over 20 unseen worlds (learners: three seeds).

Policy	Freshness ↑	Return ↑	Drones lost ↓
Masked DQN	0.146	−10.92	0.00
Masked REINFORCE	0.133	−11.49	0.00
Masked A2C	0.137	−11.20	0.00
Unmasked DQN	0.038	−65.42	1.70
Unmasked REINFORCE	0.052	−24.63	2.00
Unmasked A2C	0.037	−31.59	3.00
Greedy	0.322	−5.27	0.15
Rolling-horizon planner	0.305	−5.20	0.00
Masked random	0.145	−11.37	0.00
Random	0.095	−29.04	3.00

TABLE II
40×40 learning-rate sweep: mean freshness on 20 validation worlds (one seed). The main-study rate is in the middle column.

Agent	$\times\frac{1}{3}$	$\times 1$	$\times 3$
DQN	0.087 (3×10^{-5})	0.098 (10^{-4})	0.080 (3×10^{-4})
REINFORCE	0.072 (3×10^{-4})	0.080 (10^{-3})	0.084 (3×10^{-3})
A2C	0.084 (3×10^{-5})	0.084 (10^{-4})	0.083 (3×10^{-4})

References on the same worlds: Greedy 0.226, masked random 0.091.

V. Hyperparameter Tuning

(Written by Sayuru Bopitiya)

Tuning had two stages. At 20×20, a one-seed screen trained each agent for 200 episodes per setting and scored the final policy on five validation worlds. It chose REINFORCE's learning rate (10^{-3}, freshness 0.131 against 0.114 at 10^{-4}) and entropy weight (0, 0.132 against 0.114 at 0.01), and A2C's entropy weight (0.05, 0.164 against 0.126 at 0.01) and rollout length (32 steps). DQN scored best at 10^{-3} in this screen, but we kept 10^{-4} for the main study because the larger rate had produced the Q-value blow-up above. These settings were frozen before the 40×40 study.

Because the 40×40 rates were not tuned on the larger grid, we later trained each agent from scratch at one third, one and three times its main-study rate (one seed, 300 episodes), scoring each on 20 separate validation worlds (Table II). None of the rates closed the gap to Greedy. DQN at its main-study rate was the best fit and slightly ahead of masked random (0.098 against 0.091 freshness, -24.00 against -25.14 return), but this margin is smaller than the standard deviation across worlds (about 0.015). REINFORCE improved with a larger rate and A2C was flat over a ten-fold range. With one seed per setting, differences of about 0.01 are within seed-to-seed variation [14], so the sweep cannot rule out the learning rate as a contributing factor, only show that a three-fold change was not enough on its own.

VI. Main Study at 40×40

(Written by Sayuru Bopitiya)

TABLE III
MAIN 40×40 STUDY ON 20 UNSEEN WORLDS. MEAN ± SAMPLE SD (LEARNERS: ACROSS THREE SEED MEANS; BASELINES: ACROSS WORLDS).

Policy	Freshness ↑	Overdue ↓	Return ↑	Energy	Lost
DQN	0.084 ± 0.003	0.869	-24.79 ± 0.25	698	0
REINFORCE	0.069 ± 0.007	0.894	-25.81 ± 0.59	675	0
A2C	0.084 ± 0.012	0.869	-24.83 ± 0.92	723	0
Greedy	0.221 ± 0.027	0.656	-16.10 ± 1.50	783	0
Planner	0.221 ± 0.027	0.667	-16.55 ± 1.49	757	0
Sweep	0.065 ± 0.015	0.901	-26.07 ± 0.81	465	0
Masked random	0.084 ± 0.024	0.886	-25.46 ± 1.33	675	0
Random	0.060 ± 0.018	0.921	-42.87 ± 1.12	483	3

A. Protocol

The implementation, settings, budget and success criteria were fixed before any main-study fit started. Each agent was trained with seeds 0, 1 and 2 for 300 episodes on training worlds 0–299 (90,000 environment steps per fit). The final weights of all nine fits were then evaluated once, acting greedily over feasible actions, on 20 unseen worlds (22000–22019). Greedy, the lawnmower sweep, random and masked random flew the same worlds, giving 260 missions; the rolling-horizon planner was added later. Greedy picks the feasible joint action whose sensors would remove the most priority-weighted staleness; the planner simulates eight candidate actions four steps ahead over two sampled fire futures. An independent replay reproduced all 75,911 evaluation steps exactly, including every action, mask, state hash and reward.

B. Training behaviour

Fig. 1 shows the training returns. DQN improved steadily during the first 100 episodes, from a mean of -25.1 over the first 50 episodes to -23.8 over the last 50, and then levelled off. REINFORCE (-25.1 to -25.0) and A2C (-25.3 to -25.4) showed no trend, and their three seeds did not agree on a direction. Training freshness rose from 0.089 to 0.101 for DQN only.

C. Results on unseen worlds

Table III gives the results. All learned policies kept every drone in all 180 missions, whereas unrestricted random lost the whole fleet in all 20. Their inspection quality, however, matched masked random: DQN's freshness exceeded it by only 0.0006 and A2C's by 0.0006, while REINFORCE was 0.015 below. Greedy achieved 2.6 times the freshness of DQN and A2C. On return, DQN and A2C were about 0.65 above masked random. None of the nine fits passed the pre-registered performance gates, which required every seed to beat masked random by at least 0.01 freshness and 0.5 return, and separately to reach 90% of Greedy's freshness.

The rolling-horizon planner matched Greedy's freshness (0.2207 against 0.2208) with a slightly lower return and no losses, but needed about 66 ms per decision against 1.1 ms for Greedy in the same serial timing test. Paired by world, it beat Greedy's return in only 7 of 20 worlds, so four-step lookahead added nothing over the one-step rule here.

TABLE IV
SENSITIVITY AT 40×40: MEAN FRESHNESS ON 20 UNSEEN WORLDS, ONE SETTING CHANGED AT A TIME (LEARNERS NOT RETRAINED).

Setting	Greedy	Planner	M. rand.	DQN	REINF.	A2C
Default	0.221	0.221	0.084	0.084	0.069	0.084
Battery 120	0.205	0.208	0.083	0.096	0.083	0.094
Battery 200	0.215	0.216	0.088	0.078	0.061	0.075
Recharge 5	0.203	0.207	0.078	0.081	0.066	0.080
Recharge 12	0.226	0.227	0.087	0.085	0.071	0.085
Spread 0.02	0.208	0.216	0.080	0.084	0.069	0.084
Spread 0.08	0.208	0.214	0.082	0.086	0.066	0.084

VII. SENSITIVITY STUDY

(Written by Sayuru Bopitiya)

To test whether these conclusions depend on the chosen settings, we changed one parameter at a time and flew all policies again on the same 20 worlds (1,820 missions). The learners kept their default-setting weights, so their rows measure the robustness of a fixed policy, not what the algorithm would learn under the new setting. Table IV gives the freshness.

Greedy and the planner remained the strongest group in every condition (0.20–0.23), and all other policies stayed between 0.06 and 0.10, although rankings inside each group changed. Slower charging cost Greedy and the planner 0.014–0.018 freshness. Away from the default, Greedy lost its whole fleet in 4 of 140 missions: three low-battery drones met near the base, every move home collided, its rule fell back to "all stay" and they hovered until empty. The masked learners avoid this deadlock through the emergency return, and the planner avoided it in all of its missions. A smaller battery made the fixed learners look better (DQN 0.096) and a larger one worse (0.078); because the weights did not change, this is most likely the mask forcing earlier returns that pass near the base, not learned behaviour.

VIII. CRITICAL REVIEW

(Written by Sayuru Bopitiya)

A. Was the problem solved?

Only partly. The engineering goals were met and every learned policy flew full missions without losing a drone, but the learning goal was not: the learned policies inspected no better than random feasible actions, and a one-step heuristic was more than twice as good. The zero-loss result belongs to the mask, since masked random, which learns nothing, also lost no drones. Without that control, safety could easily have been misread as learned competence.

B. Why the learners stayed at masked-random level

All three agents learn from differences in reward between actions. To measure how large these are, we froze the state every tenth step under random feasible play, tried every feasible joint action, and recorded the immediate reward. At 40×40 the best action beats the second-best by only 0.0005 on average, and the standard deviation across actions is 0.002. The standard deviation of the mission return across worlds is

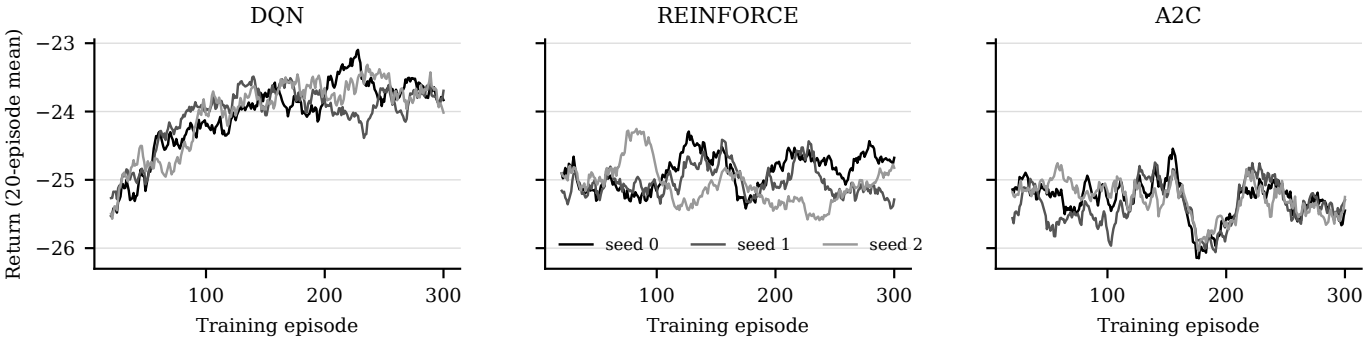


Fig. 1. Training return of the nine main-study fits at 40×40 (20-episode moving mean; returns include exploration).

0.87, about 430 times larger (130 times at 20×20). The useful signal is therefore buried in noise: REINFORCE updates once per episode from a single noisy return; A2C bootstraps from a critic that must itself resolve these tiny differences; and DQN must estimate Q-values whose gaps between actions are smaller than its approximation error. This matches the flat learning curves of the two policy-gradient agents, and the fact that DQN's training gains did not transfer to unseen worlds.

The joint action space makes this worse. One drone's good move is rewarded together with the other two drones' choices, so the agent cannot easily tell which drone deserved the credit. Greedy, by contrast, computes exactly the quantity the reward measures, so it does not need to learn it from noisy samples. Three seeds and 90,000 steps per fit are also a small budget for deep RL, where results vary strongly between seeds [14]. However, the learning-rate sweep and the DQN plateau suggest that more compute alone would not close a gap of this size.

C. Implications and validity

A well-designed heuristic with full information is a strong baseline for fleet inspection, and safety obtained through masking should not be reported as learning success. The conclusions hold for this simulator only: fire, wind and battery values are illustrative, the state is fully observed, and the 20×20 and 40×40 studies differ in several settings at once, so their difference is not a controlled scaling result. Part II proposes changes that target the measured causes.

Part II: Improvements and Future Work

IX. Limitations and Potential Improvements

(Written by all members; each subsection names its author)

Part I identified three limitations of the current work, each supported by our own results:

- L1 *Credit assignment over a joint action.* One network scores all 125 joint actions, so a single scalar reward must teach it which of three drones acted well. The learned policies matched masked random on unseen worlds (Table III).
- L2 *A weak, noisy learning signal.* The immediate reward separates the best and second-best actions by about 0.0005, while mission returns vary 430 times more than rewards across actions; the policy-gradient learning curves were flat (Fig. 1).
- L3 *Learning from scratch with a small budget.* Every agent started from random weights with 90,000 steps, although a strong controller (Greedy, 0.221 freshness) was available; DQN's training gains levelled off after about 100 episodes.

We propose one structural improvement for each. None of them is a change of hyperparameters. Each states the change, the literature that supports it, how it would be evaluated and its main risks.

A. Improvement 1: Factorised credit assignment (L1)

(Written by Lithma Perera)

Proposal. Replace the 125-way joint head with per-drone utility networks that share the spatial encoder, and combine them into a joint value with a learned mixing network, as in QMIX [15]. Each drone's utility depends on its own position and battery plus the shared fire and age maps, and the mixer is conditioned on the full state. Because the mixer is monotonic in each drone's utility, the joint greedy action can be found by maximising each drone's utility separately, so the output grows as $5N$ rather than 5^N. The feasibility mask would be applied per drone, with the collision check kept as a joint filter. A policy-gradient alternative is a centralised critic with a counterfactual baseline (COMA), which compares the joint value with the value obtained when only one drone's action is replaced by an average over its alternatives [16].

Justification. Both methods were designed for cooperative teams that share one reward, which is exactly our situation. QMIX outperformed independent learners and simpler value decompositions on cooperative StarCraft micromanagement tasks [15], and COMA's counterfactual advantage reduced the variance caused by teammates' actions in the same domain [16]. In our task, a drone that inspects a stale fire front while another wanders currently receives the same scalar signal as both; a per-drone utility can attribute the gain to the drone whose footprint caused it. Factorisation also scales: with five drones a joint head would need 3,125 outputs, while per-drone utilities need only 25.

Evaluation. Keep the environment, mask, budget, seeds and world split of the main study and change only the action and

value structure. Report freshness, the number of cells inspected by more than one drone at once, charger waiting time and the gap to Greedy. If freshness stays at masked-random level, credit assignment is not the main bottleneck, which would itself be a useful result.

Expected effect and risks. The simplest factorisation, a sum of per-drone values (VDN) [17], cannot represent cases where one drone's best move depends on the others, such as two drones heading for the same fire front; QMIX's state-conditioned mixer relaxes this while keeping decentralised maximisation [15]. Monotonic mixing still cannot express every coordination pattern, for example when two drones are each useful alone but wasteful together, so duplicated inspection is the diagnostic to watch. Charger contention is the case most likely to need the joint collision filter we would keep.

B. Improvement 2: Potential-based shaping for inspection urgency (L2)

(Written by Sulakna Weerasinghe)

Proposal. Add a training-only shaping term $F(s, a, s') = \gamma\,\Phi(s') - \Phi(s)$ with a potential that measures how well placed the fleet is to serve the most urgent locations:

$$\Phi(s) = -\lambda \sum_{k=1}^{3} \min_{i \in U(s)} d(x_k, i) - \mu \sum_{k=1}^{3} \max\big(0,\, d(x_k, \text{base}) - \beta_k\big), \tag{4}$$

where $U(s)$ are the most urgent cells (high weight and close to their deadline), d is grid distance, x_k and β_k are drone k's position and the distance its battery allows, and λ, μ are small scales. The first term rewards moving towards urgent cells before they are reached; the second rewards keeping a safe return margin. Evaluation always uses the unshaped reward in (3).

Justification. Ng et al. proved that adding $\gamma\Phi(s') - \Phi(s)$ leaves the set of optimal policies unchanged, whereas arbitrary shaping can create loops that earn reward without completing the task [8]. The term is useful precisely where our reward is weakest: today a drone receives nothing until its sensor reaches a stale cell, so the ten steps it takes to get there are credited only through noisy returns. The potential difference rewards each of those steps directly, which increases the gap between good and poor actions without changing what is optimal. For our finite horizon, the potential must be set to zero at termination so that ending early cannot earn extra credit.

Evaluation. Repeat the reward-signal measurement of Section VIII with the shaped reward to check that the gap between the best and second-best actions grows. Then train with and without shaping under the main-study protocol and compare learning speed and unshaped test freshness. A higher shaped return without higher test freshness would mean the potential is poorly chosen.

Expected effect and risks. Potential-based shaping is equivalent to initialising the value function with Φ [18], so a good potential should mostly speed up early learning, where our DQN curve shows its only gains. A poor potential cannot change the optimal policy, but it can slow learning by pointing exploration the wrong way, for example towards an urgent cell that another drone is already serving. The urgency set $U(s)$ should therefore exclude cells inside another drone's footprint, and the scales λ and μ should be fixed on validation worlds before any test run.

C. Improvement 3: Learning from demonstrations (L3)

(Written by Sayuru Bopitiya)

Proposal. Start DQN from Greedy's behaviour instead of from random weights, using Deep Q-learning from Demonstrations (DQfD) [19]. Greedy and planner missions on training worlds fill a demonstration buffer that is never overwritten. The network is first pre-trained on it with the usual TD loss plus a large-margin loss that makes the demonstrated action score higher than the others, then continues learning from its own experience while still sampling demonstrations. Prioritised replay [20] would sample rare but important transitions, such as charger hand-overs and late returns, more often than routine patrol steps.

Justification. DQfD started from much better performance than DQN on almost all Atari games it was tested on and exceeded its demonstrator on several, because the TD loss allows the agent to improve on demonstrations rather than only copy them [19]. Prioritised replay improved DQN on most Atari games by replaying transitions with large TD errors [20]. Our own 20×20 imitation experiments showed that copying Greedy alone generalised poorly (9–18% agreement on new worlds), which is the weakness DQfD addresses by combining imitation with RL on the agent's own states.

Evaluation. Compare DQfD with plain DQN under the same step budget, counting demonstration steps against it. Plot freshness against steps to measure sample efficiency, and test whether the final policy exceeds Greedy on unseen worlds, matches it, or falls back towards masked random once demonstrations are diluted.

Expected effect and risks. Pure imitation suffers from compounding errors: one mistake leads to states the demonstrator never visited, where the copy has no guidance [21]. This explains why our supervised copies of Greedy fitted training decisions well but generalised poorly. DQfD reduces the problem because its TD loss is also trained on the agent's own states. The main risk is that the agent simply reproduces Greedy; reaching Greedy's freshness with zero losses would already be a large improvement on the current result, and any gain beyond it would show what RL adds to the heuristic.

D. Order of work

The three improvements differ in cost. Shaping (I2) changes only the reward used in training and can be checked cheaply with the reward-signal measurement before any training. DQfD (I3) reuses the existing DQN and only adds a demonstration buffer and a margin loss. Factorisation (I1) needs a new network and changes to the mask, so it is the most expensive. We would run them in that order and combine the ones that help, since they target different limitations: I2

strengthens the signal, I3 provides a good starting point and I1 divides the credit. Each step would follow the main-study protocol, so that any gain can be attributed to the change alone.

X. Future Work

(Written by all members; each subsection names its author)

The following directions do not repair the limitations above. They extend the problem towards real fire operations with methods we have not used. Each changes the MDP itself and builds on published work.

A. Extension 1: Partial observability and belief maps

(Written by Lithma Perera)

Our controller sees the true fire map at every step. In practice, fire is known only where a drone or other sensor has recently looked, and smoke causes missed and false detections. We would hide the fire state, model a camera with detection and false-alarm rates, and require the fleet to keep a belief map of where fire is likely to be. The task then becomes a partially observable MDP (POMDP), and the controller must trade inspecting known fire against searching areas where the belief is uncertain. Exact POMDP solutions are intractable for state spaces of this size, and Hauskrecht reviews value-function approximations that make them practical [22]. Julian and Kochenderfer showed that deep RL can learn distributed wildfire surveillance when each aircraft acts on a belief map of the fire built from its own observations [4]. In MDP terms, the state stays the same but the agent receives an observation $o_t \sim p(o \mid s_t)$ instead of s_t, the fire and inspection maps in the input are replaced by belief and uncertainty maps, and the reward can add a term for reducing uncertainty. We would compare a belief-based learner with an information-gain planner and the Greedy rule applied to the belief, and report detection delay, belief error and freshness measured against the true fire.

B. Extension 2: Decentralised fleets with limited communication

(Written by Sulakna Weerasinghe)

We assume one controller with instant access to every drone. A larger fleet would more likely have each drone deciding from its own sensors and exchanging short messages over an unreliable link. We would give each drone a local view, add a communication channel with bandwidth limits, delay and dropouts, and model the problem as a decentralised POMDP. Foerster et al. showed that agents can learn useful communication protocols end to end, by reinforcement or by back-propagating through the channel during centralised training [23]. Centralised training with decentralised execution, as in QMIX [15], keeps the advantage of a shared critic during training while removing the need for a central controller in flight. Each drone would choose among its own five commands, so the action space grows linearly with the fleet, and the shared reward would be kept so that the agents remain cooperative. The study would measure freshness and safety as message rates fall and links fail, against a baseline in which drones share nothing, and would also test fleets larger than three, which a joint action space cannot reach.

TABLE V
Summary of proposed improvements and extensions.

	Change	Success measure
I1	Per-drone utilities with a mixing network or counterfactual critic [15], [16]	Higher test freshness; less duplicated inspection
I2	Potential-based shaping towards urgent cells [8]	Larger action gap; higher unshaped freshness
I3	DQfD with prioritised replay of Greedy missions [19], [20]	Faster learning; freshness at or above Greedy
E1	Hidden fire and belief maps (POMDP) [4], [22]	Detection delay and belief error
E2	Local views and learned communication [23]	Freshness and safety under link loss
E3	Calibrated fire, mixed fleets, domain randomisation [5], [24]	Robustness on held-out regions

C. Extension 3: Calibrated and heterogeneous operations

(Written by Sayuru Bopitiya)

Our fire model, terrain and battery values are illustrative. To move towards operational use, we would drive the simulator with real terrain and fuel maps, calibrated spread under recorded weather, drones with different endurance and sensors, and several charging sites. Policies would be trained with domain randomisation over these parameters, which Tobin et al. used to transfer models trained only on simulated images to a real robot [24], and tested on held-out regions and weather regimes. Viseras et al. provide a precedent for monitoring a fire front with several UAVs using deep Q-learning [5]. The transition model in (1) would be replaced by a calibrated spread model, and the reward weights would be set with fire managers so that they reflect real priorities. Results would report freshness of the actual front, energy, robustness to sensor and drone failures, and decision latency, which our planner results show can become the limiting factor: the planner matched Greedy but needed about 60 times longer per decision.

D. Relevance to the project goals

The project asked whether learning can coordinate a battery-limited fleet better than conventional planning. The improvements test that question more fairly on the present task; the extensions test whether the answer changes when the task becomes more realistic. Partial observability (E1) removes the full information that makes Greedy strong, so learned memory and belief may matter more there than here. Decentralisation (E2) removes the central controller that the current comparison takes for granted. Calibration (E3) checks whether any advantage survives realistic fire behaviour. Together they would show where RL offers a useful trade-off for fleet inspection, which was the outcome promised in our proposal.

Table V summarises the six proposals. Any of these studies should follow the discipline of our main study: fix settings and success criteria before training, keep test worlds unseen, use several seeds, and report negative results.

PART III: STATEMENT OF CONTRIBUTIONS

XI. STATEMENT OF CONTRIBUTIONS

(Written by Lithma Perera, Sulakna Weerasinghe and Sayuru Bopitiya)

All group members contributed equally to this project.

Table VI lists the author of each section of the project notebook and of this report. All three members reviewed the complete code, notebook and report.

TABLE VI
SECTION AUTHORSHIP.

Member	Notebook sections	Report sections
Lithma Perera	1–4: setup, problem, MDP, environment	I, II; Improvement 1, Extension 1
Sulakna Weerasinghe	5–8: baselines and mask, RL agents, implementation check, 20×20 development	III, IV; Improvement 2, Extension 2
Sayuru Bopitiya	9–12: tuning, main study, sensitivity, critical review	V–VIII; Improvement 3, Extension 3
All members	13: references	Abstract; introductions of IX and X; XI

REFERENCES

[1] D. M. J. S. Bowman, G. J. Williamson, J. T. Abatzoglou, C. A. Kolden, M. A. Cochrane, and A. M. S. Smith, "Human exposure and sensitivity to globally extreme wildfire events," *Nature Ecology & Evolution*, vol. 1, no. 3, Art. no. 0058, 2017, doi: 10.1038/s41559-016-0058.

[2] C. Yuan, Y. Zhang, and Z. Liu, "A survey on technologies for automatic forest fire monitoring, detection, and fighting using unmanned aerial vehicles and remote sensing techniques," *Canadian Journal of Forest Research*, vol. 45, no. 7, pp. 783–792, 2015, doi: 10.1139/cjfr-2014-0347.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[4] K. D. Julian and M. J. Kochenderfer, "Distributed wildfire surveillance with autonomous aircraft using deep reinforcement learning," *Journal of Guidance, Control, and Dynamics*, vol. 42, no. 8, pp. 1768–1778, 2019, doi: 10.2514/1.G004106.

[5] A. Viseras, M. Meissner, and J. Marchal, "Wildfire front monitoring with multiple UAVs using deep Q-learning," *IEEE Access*, 2021, doi: 10.1109/ACCESS.2021.3055651.

[6] L. Perera, S. Weerasinghe, and S. Bopitiya, "Wildfire drone fleet: environment, agents and experiment records," COMP6008 project notebook `Wildfire_Drone_Fleet_clean.ipynb` and code, Curtin University, Oct. 2026.

[7] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.

[8] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. 16th Int. Conf. Machine Learning (ICML)*, 1999, pp. 278–287.

[9] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992, doi: 10.1007/BF00992696.

[10] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, PMLR vol. 48, 2016, pp. 1928–1937.

[11] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. 32nd AAAI Conf. Artificial Intelligence*, 2018, pp. 2669–2678.

[12] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *Proc. 35th Int. Florida Artificial Intelligence Research Society Conf. (FLAIRS)*, 2022.

[13] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th AAAI Conf. Artificial Intelligence*, 2016, pp. 2094–2100.

[14] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. 32nd AAAI Conf. Artificial Intelligence*, 2018, pp. 3207–3214.

[15] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, PMLR vol. 80, 2018, pp. 4295–4304.

[16] J. N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proc. 32nd AAAI Conf. Artificial Intelligence*, 2018, pp. 2974–2982.

[17] P. Sunehag *et al.*, "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proc. 17th Int. Conf. Autonomous Agents and MultiAgent Systems (AAMAS)*, 2018, pp. 2085–2087.

[18] E. Wiewiora, "Potential-based shaping and Q-value initialization are equivalent," *Journal of Artificial Intelligence Research*, vol. 19, pp. 205–208, 2003, doi: 10.1613/jair.1190.

[19] T. Hester *et al.*, "Deep Q-learning from demonstrations," in *Proc. 32nd AAAI Conf. Artificial Intelligence*, 2018, pp. 3223–3230.

[20] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *Proc. 4th Int. Conf. Learning Representations (ICLR)*, 2016.

[21] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, PMLR vol. 15, 2011, pp. 627–635.

[22] M. Hauskrecht, "Value-function approximations for partially observable Markov decision processes," *Journal of Artificial Intelligence Research*, vol. 13, pp. 33–94, 2000, doi: 10.1613/jair.678.

[23] J. N. Foerster, Y. M. Assael, N. de Freitas, and S. Whiteson, "Learning to communicate with deep multi-agent reinforcement learning," in *Advances in Neural Information Processing Systems 29 (NeurIPS)*, 2016, pp. 2137–2145.

[24] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.

APPENDIX A
UPDATE EQUATIONS

(Written by Sulakna Weerasinghe)

Let $\mathcal{A}_{\mathrm{f}}(s)$ be the feasible joint actions in state s and $m(s) \in \{0,1\}^{125}$ its mask. Masked scores replace infeasible entries by the lowest finite value, so $\pi_\theta(a|s) = 0$ for $a \notin \mathcal{A}_{\mathrm{f}}(s)$.

A. DQN

For a minibatch $\mathcal{B}$ of transitions (s, a, r, s', d) with terminal flag d,

$$y = r + \gamma\,(1-d)\max_{a' \in \mathcal{A}_{\mathrm{f}}(s')} Q_{\bar{\theta}}(s', a'), \tag{5}$$

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{B}|}\sum_{\mathcal{B}} \mathrm{Huber}\big(Q_\theta(s,a) - y\big), \tag{6}$$

with $\bar{\theta} \leftarrow \theta$ every 100 environment steps and the ϵ-greedy action drawn uniformly from $\mathcal{A}_{\mathrm{f}}(s)$.

B. REINFORCE with a learned baseline

For one episode of length T with reward-to-go $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$,

$$\mathcal{L}(\theta,\phi) = -\frac{1}{H}\sum_{t=0}^{T-1} \gamma^t \log \pi_\theta(a_t|s_t)\big(G_t - V_\phi(s_t)\big) + \frac{c_v}{T}\sum_{t=0}^{T-1}\big(G_t - V_\phi(s_t)\big)^2, \tag{7}$$

where the baseline enters the actor term as a constant and $c_v = 0.5$. Subtracting $V_\phi(s_t)$ leaves the gradient unbiased because $\mathbb{E}_{a\sim\pi}[\nabla_\theta \log \pi_\theta(a|s)\, b(s)] = b(s)\nabla_\theta \sum_a \pi_\theta(a|s) = 0$ [3], [9].

C. A2C

For a segment of $n = 32$ steps starting at t_0,

$$G_t^{(n)} = \sum_{k=t}^{t_0+n-1} \gamma^{k-t} r_k + \gamma^{t_0+n-t}(1-d)\, V_\phi(s_{t_0+n}), \tag{8}$$

$$\mathcal{L} = -\frac{1}{n}\sum_t \gamma^t \log \pi_\theta(a_t|s_t)\, A_t + c_v\, \overline{(G_t^{(n)} - V_\phi(s_t))^2} - c_e\, \overline{\mathcal{H}[\pi_\theta(\cdot|s_t)]}, \tag{9}$$

with $A_t = G_t^{(n)} - V_\phi(s_t)$ treated as a constant in the actor term, $c_v = 0.5$, $c_e = 0.05$, and bars denoting means over the segment [10].

D. Fire-spread probability

Each burning neighbour j of an unburnt cell i ignites it independently with probability $p_{ji} = \min(1, q\,\omega_{ji}\,\tau_i)$. The cell therefore stays unburnt only if every neighbour fails, with probability $\prod_j (1 - p_{ji})$, which gives (1).